Checkpoint 5 — Implementação do Aluno 1

Cache local LFU de chunks no Peer | explicação de responsabilidades,

fluxo, decisões e evidências

GUIA EXPLICATIVO

Fluxos, módulos, funções, decisões, evidências e pendências

Leitura baseada no código e na documentação do repositório

Revisão: 05/11/2026

Uso didático: implementação atual separada de requisitos e trabalho futuro

1. Escopo desta parte

No C5, a parte do Aluno 1 implementa cache local de chunks comprimidos no armazenamento do Peer. A finalidade é evitar leituras repetidas do disco quando downloads solicitam novamente o mesmo chunk. O cache atua no caminho de leitura do Peer; não substitui os arquivos persistidos e não altera o formato das mensagens de rede.

Arquivos principais: storage.c e storage.h implementam o cache e sua API; peer.c usa essa API ao atender M_DOWNLOAD_REQ e registra hit, miss e frequência. Os testes locais estão em tests/c5/test_cache_lfu.c; o teste de integração de rede está em tests/c5/transaction_integration.py.

2. Dados armazenados e chave

Cada item em memória contém o ObjectID do documento, o índice do chunk, os bytes comprimidos e o descritor necessário para servir o chunk. A identidade da entrada é o par (ObjectID, índice), portanto chunks diferentes do mesmo documento ocupam entradas diferentes.

O cache guarda os bytes comprimidos já persistidos. Em um hit, o Peer evita a leitura do arquivo do chunk em objects e pode responder usando a cópia em memória. A resposta continua passando pelo protocolo normal, com descritor e validações previstas no caminho de transferência.

3. Configuração de capacidade

PEER_CACHE_BYTES define o limite de bytes de cache por processo Peer. O padrão é 16 MiB; valor zero desativa o armazenamento no cache. A capacidade considera o tamanho das cópias dos chunks, e a implementação também trata a reconfiguração sem deixar entradas incompatíveis com o limite atual.

Essa configuração é local ao Peer. Ela não é enviada ao Super Peer e não modifica tests/config/c5.conf.

4. Política LFU e evicção

Em cada acesso, a frequência da entrada é atualizada. Quando uma nova cópia não cabe no limite, a implementação escolhe uma entrada de menor frequência (Least Frequently Used). Se houver empate, remove a que está há mais tempo sem acesso, usando o instante de último uso como critério LRU de desempate.

A remoção afeta apenas a cópia em memória. Os arquivos de chunks e manifests duráveis continuam em objects; uma leitura posterior de uma entrada removida volta ao armazenamento persistente e pode popular o cache novamente. Com capacidade zero, a leitura segue pelo armazenamento sem reter uma entrada.

5. Caminho de leitura no Peer

- O cliente envia M_DOWNLOAD_REQ com ObjectID e índice do chunk.
- peer.c chama storage_read_chunk_with_cache_info, que procura a chave composta no cache.

- Em caso de hit, recebe uma cópia de bytes pertencente ao chamador e atualiza a frequência/recência.

- Em caso de miss, a função lê o chunk durável e, se houver capacidade, conserva uma cópia comprimida no cache.

- O Peer responde pelo protocolo de transferência já existente e libera a cópia temporária conforme a API.

- O log do Peer informa LFU CACHE HIT ou MISS, o índice e a frequência observada. A evicção registra o índice e a frequência removidos.

O chamador recebe memória própria, não um ponteiro interno que poderia ser invalidado por uma evicção concorrente. O estado e as estatísticas do cache são protegidos para acesso concorrente entre workers.

6. API e estatísticas

storage_read_chunk_with_cache_info executa a leitura e retorna ao chamador os dados, tamanho, descritor e informações de hit/miss e frequência. StorageCacheStats, exposta por storage_get_cache_stats, agrega informações do cache sem exigir que a camada de armazenamento exponha suas estruturas privadas.

As estatísticas de frequência e hit/miss ajudam a observar o comportamento do cache, enquanto os logs de evicção mostram qual entrada foi removida. A API local de armazenamento é distinta da integração TCP: o teste unitário cobre diretamente o módulo; o teste de integração verifica o comportamento através de Peer e cliente reais.

7. Testes e evidências

Teste local

make test-c5 compila com C2x e avisos estritos tratados como erros e executa test_cache_lfu. O resultado registrado em 05/11/2026 foi aprovado. Os cenários cobertos incluem:

- miss inicial, hit subsequente e aumento de frequência;

- escolha LFU e desempate pela recência;

- capacidade insuficiente e capacidade zero;

- leitura novamente a partir do armazenamento durável depois da evicção;

- reconfiguração concorrente do cache.

Integração pela rede

tests/c5/transaction_integration.py, executado pelo alvo make test-c5, faz dois downloads do mesmo chunk via TCP, compara os arquivos baixados byte a byte e verifica no log do Peer o MISS seguido de HIT com frequência 2. A saída reportou sucesso para o cenário integrado de C5, incluindo também verificações do protocolo de transações do Super Peer.

O teste local demonstra as regras da API de cache; o teste TCP demonstra que o caminho real do download usa o cache. Os dois níveis são complementares.

8. Limites e leitura correta do resultado

O cache é volátil e local ao processo: reiniciar um Peer remove as entradas de memória, e o conteúdo durável continua sendo a fonte de recuperação. A evidência C5 comprova os cenários citados, mas não representa benchmark de throughput nem prova formal de ausência de condições de corrida para toda carga possível.

Esta documentação descreve a parte do Aluno 1. O journal durável, o 2PC e a transferência incremental de estado do Super Peer pertencem à parte do Aluno 2 e estão explicados no guia C5 correspondente.